Solving Boundary Value Problems with Chebfun

Spectral Methods, Worked Examples, and Numerical Experiments

Jiaqi Ge
Guangdong Technion – Israel Institute of Technology
Advised by Prof. Antti Rasila

November 2025

Abstract

This report studies boundary value problems (BVPs) and their numerical solution using Chebfun, a MATLAB system for numerical computing with functions. We first review the theory of BVPs — existence, uniqueness, and stability, and how these differ from initial value problems — together with the classical analytical tools (Green's functions, Sturm–Liouville theory, and the maximum principle) and the main families of numerical methods. We then introduce Chebfun and its `chebop` interface for differential operators. The core of the report is a set of eight worked examples (nonlinear BVPs, coupled systems, singular Sturm–Liouville problems, and classical ODE dynamics) followed by four quantitative experiments that measure what Chebfun actually delivers: spectral versus second-order convergence, eigenvalue accuracy, solution non-uniqueness in the Bratu problem, and automatic resolution of boundary layers. All numerical results in this report were produced by the accompanying code, which is available at https://github.com/Gracegejiaqi/bvp-chebfun.

Contents

1 Introduction to Boundary Value Problems

A boundary value problem (BVP) is a differential equation whose solution is constrained at more than one point. A representative second-order linear example is

$$y''(x) + p(x)\,y'(x) + q(x)\,y(x) = g(x), \qquad x \in [0,1], \tag{1}$$

together with the boundary conditions

$$y(0) = \alpha, \qquad y(1) = \beta. \tag{2}$$

1.1 Existence and uniqueness

For initial value problems (IVPs), the Picard–Lindelöf theorem gives a clean sufficient condition: if $f(t,y)$ is continuous in t and Lipschitz in y on a neighbourhood of (t_0, y_0), then $y' = f(t,y)$, $y(t_0) = y_0$ has a unique local solution. For BVPs the situation is more subtle. A BVP may have

(i) a unique solution,

(ii) multiple solutions, or

(iii) no solution.

From a numerical perspective, case (ii) is usually not fatal — algorithms converge to *one* solution, and a good initial guess selects which one (we demonstrate this with the Bratu problem in section 4.3). Case (iii) is more dangerous: even when the target problem is solvable, nearby problems generated during discretization or iteration might not be.

1.2 Stability: IVPs versus BVPs

For a linear system $y' = Ay$, the stability of the equilibrium $y = 0$ is determined entirely by the eigenvalues of A: asymptotic stability iff every eigenvalue has negative real part. Stability of a BVP, however, depends on how the boundary conditions interact with the system's modes. Consider, for $c > 0$,

$$y' = Ay, \qquad A = \begin{bmatrix} -c & 0 \\ 0 & c \end{bmatrix}, \qquad y_1(0) = 1, \quad y_2(b) = 1.$$

As an IVP with data at $x = 0$ this is unstable — the second component grows like e^{cx}. But *as a BVP*, fixing y_2 at the right endpoint b turns the growing mode into a decaying one (it effectively integrates backward), and the problem is well-conditioned. The same operator can therefore be stable or unstable depending purely on where the data is imposed.

1.3 Why BVPs are harder than IVPs

BVPs are generally more challenging because of their:

- **Global nature:** constraints must hold simultaneously at different points, so the solution cannot be marched out step by step.
- **Non-uniqueness or nonexistence** of solutions.
- **Numerical instability** of naive shooting methods, where small errors in the initial slope are amplified exponentially.
- **Boundary layers** and sensitivity to parameters (section 4.4).

1.4 Classical analytical results

Green's functions. Once the Green's function $G(x, \xi)$ of a linear operator L is known, the non-homogeneous problem $L[u] = f$ with homogeneous boundary conditions is solved in closed form by

$$u(x) = \int_a^b G(x, \xi)\, f(\xi)\, d\xi.$$

The function $G(x, \xi)$ encodes the response at x to a unit point source at ξ, and its boundedness is tied to the well-posedness of the BVP.

Sturm–Liouville theory. A regular Sturm–Liouville problem has the form

$$(p(x)y')' - q(x)y + \lambda r(x)y = 0, \qquad \alpha_1 y(0) + \alpha_2 y'(0) = 0, \quad \beta_1 y(1) + \beta_2 y'(1) = 0.$$

Its eigenvalues are real and form an increasing sequence $\lambda_1 < \lambda_2 < \cdots \to \infty$, and the corresponding eigenfunctions are orthogonal with respect to the weight $r(x)$ and complete in L^2. We verify this spectral structure numerically in section 4.2.

Maximum principle. If u satisfies $L[u] = u'' + g(x)u' \geq 0$ on (a, b) with g bounded, then u attains its maximum only on the boundary; if it attains an interior maximum equal to its supremum, it is constant. This provides qualitative control of solutions independent of any explicit formula.

1.5 Numerical methods overview

Common numerical approaches to BVPs include:

- **Shooting methods** — recast the BVP as an IVP and root-find on the unknown initial data.

- **Finite difference methods** — discretize derivatives on a grid; typically second-order accurate, $O(h^2)$.
- **Finite element methods** — solve a weak form with piecewise polynomial bases.
- **Spectral methods** — expand in global polynomials for very high accuracy; this is the family Chebfun belongs to.

Section 4.1 contrasts finite differences with Chebfun's spectral collocation directly.

2 Chebfun

Chebfun is an open-source MATLAB system for numerical computing with functions rather than numbers. Its foundation is piecewise polynomial interpolation in Chebyshev points: a smooth function is represented by a polynomial of adaptively chosen degree, accurate to machine precision. Chebfun then provides continuous analogues of the usual array operations — integration, differentiation, rootfinding, and the solution of differential equations.

For BVPs the key object is the `chebop`, a differential operator with attached boundary conditions:

- `N.op` holds the (possibly nonlinear) differential operator;
- `N.lbc`, `N.rbc`, and `N.bc` hold the boundary conditions, which may themselves be nonlinear;
- the backslash `u = N \ rhs` solves $L[u] = \text{rhs}$, using a damped Newton iteration for nonlinear problems.

Two features matter throughout this report. First, **spectral accuracy**: solutions are typically obtained to near machine precision with a small number of degrees of freedom. Second, **automatic adaptivity**: Chebfun chooses the polynomial degree needed for convergence, so the user never refines a grid by hand. We quantify both below.

3 Worked Examples

This section solves eight problems with Chebfun. Each example states the problem, gives the essential code, and shows the computed solution.

3.1 Example 1: Nonlinear BVP with a nonlinear boundary condition

We solve

$$u''(x) + u'(x)\, e^{u(x)} = \mu \sin(2\pi x), \qquad x \in [0, 1],$$

with $u(0) = 1$ and the *nonlinear* right boundary condition $u'(1) + u(1)^3 = 0$, for $\mu = 12$.

```
mu = 12; dom = [0, 1];
x = chebfun('x', dom);
N = chebop(dom);
N.op = @(x, u) diff(u, 2) + diff(u).*exp(u);
N.lbc = 1; % u(0) = 1
N.rbc = @(u) diff(u) + u.^3; % u'(1) + u(1)^3 = 0
u = N \ (mu * sin(2*pi*x));
```

Chebfun handles the nonlinear boundary condition with no special treatment (fig. 1).

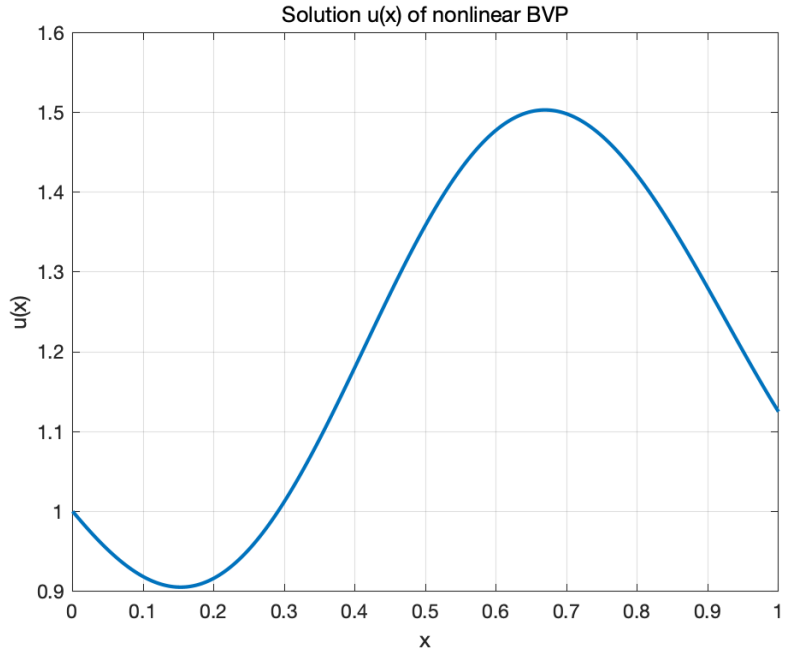

Figure 1: Solution of the nonlinear BVP with $\mu = 12$.

3.2 Example 2: Nonlinear homogeneous BVP and coefficient decay

For $y'' + y^3 = 0$ with $y(0) = 0$, $y(1) = 1$, we plot both the solution and the decay of its Chebyshev coefficients (fig. 2). The rapid coefficient decay is the signature of spectral accuracy: the solution is resolved to machine precision by a polynomial of modest degree.

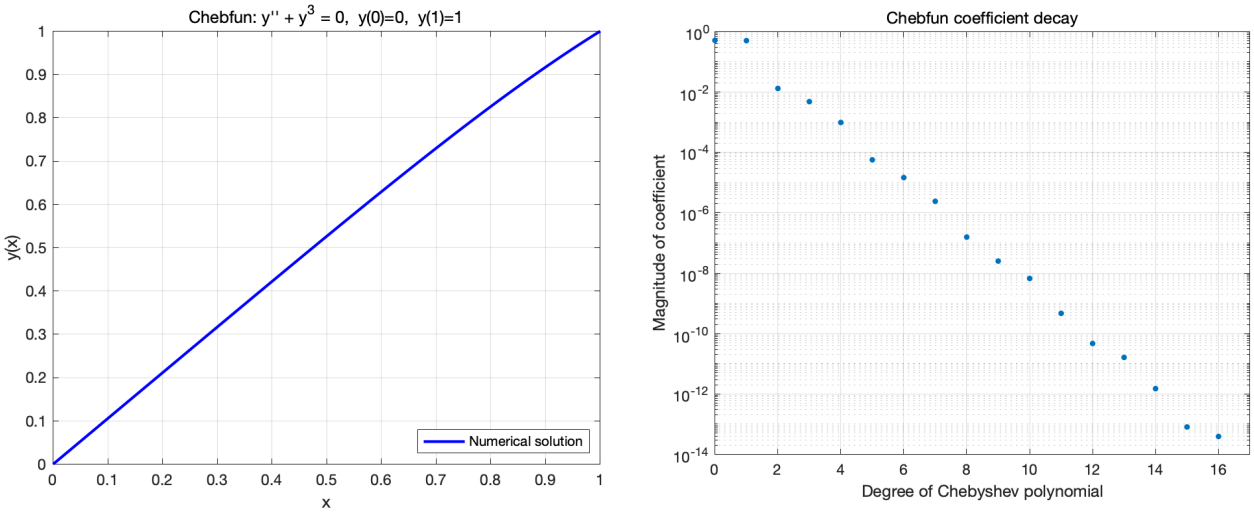

Figure 2: Left: solution of $y'' + y^3 = 0$. Right: magnitude of the Chebyshev coefficients, decaying to machine precision.

3.3 Example 3: Nonlinear inhomogeneous BVP

We solve $v'' = 3v + x^2 + 10v^3$ on $[0, 1]$ with $v(0) = v(1) = 0$. The cubic term makes the problem genuinely nonlinear; Newton's iteration converges from the zero initial guess (fig. 3).

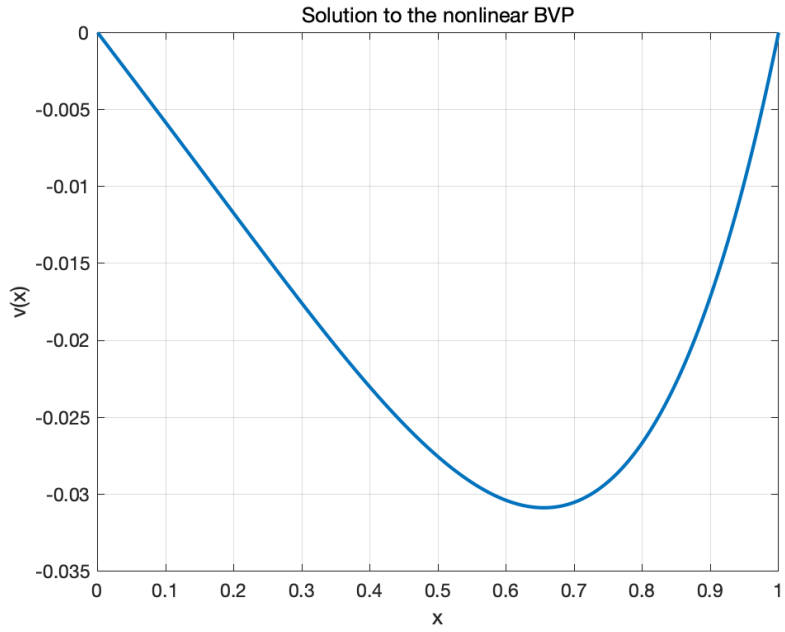

Figure 3: Solution of the nonlinear inhomogeneous BVP.

3.4 Example 4: A coupled nonlinear system

The original course example imposed four boundary conditions on a system of total order two, making it over-determined and ill-conditioned. We replace it with a well-posed second-order system on $[0, 1]$:

$$u_1'' = u_1 u_2 + \sin(\pi x), \qquad u_2'' = -u_2 + u_1^2,$$

with $u_1(0) = 0,\ u_1(1) = 1,\ u_2(0) = 1,\ u_2(1) = 0$. Two second-order equations give a fourth-order system, matched exactly by the four Dirichlet conditions.

```
N = chebop([0, 1]);
N.op = @(x, u) [ diff(u{1},2) - u{1}.*u{2} - sin(pi*x);
                 diff(u{2},2) + u{2} - u{1}.^2 ];
N.lbc = @(u) [ u{1}; u{2} - 1 ]; % u1(0)=0, u2(0)=1
N.rbc = @(u) [ u{1} - 1; u{2} ]; % u1(1)=1, u2(1)=0
u = N \ [0; 0];
```

The solver converges to $17 + 17$ degrees of freedom, and substituting the solution back into the equations gives residuals of 7.6×10^{-10} and 2.8×10^{-10} (fig. 4).

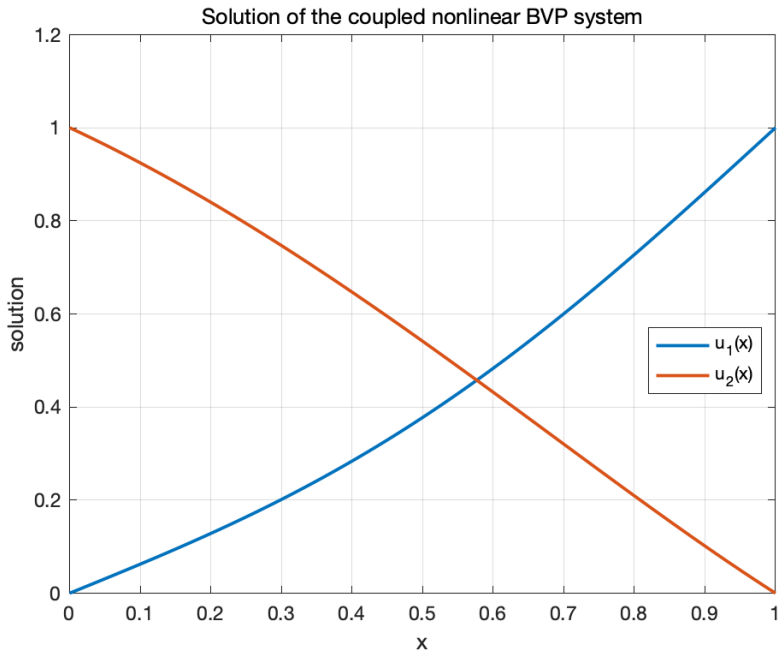

Figure 4: Solution of the coupled nonlinear BVP system.

3.5 Example 5: Exact versus numerical solution

For $y'' + 4y = 0$ with $y(0) = -2$, $y(\pi/4) = 10$, the exact solution is $y(x) = -2\cos(2x) + 10\sin(2x)$. Comparing it with the Chebfun solution gives a maximum pointwise error of 7.8×10^{-14} — essentially

machine precision (fig. 5).

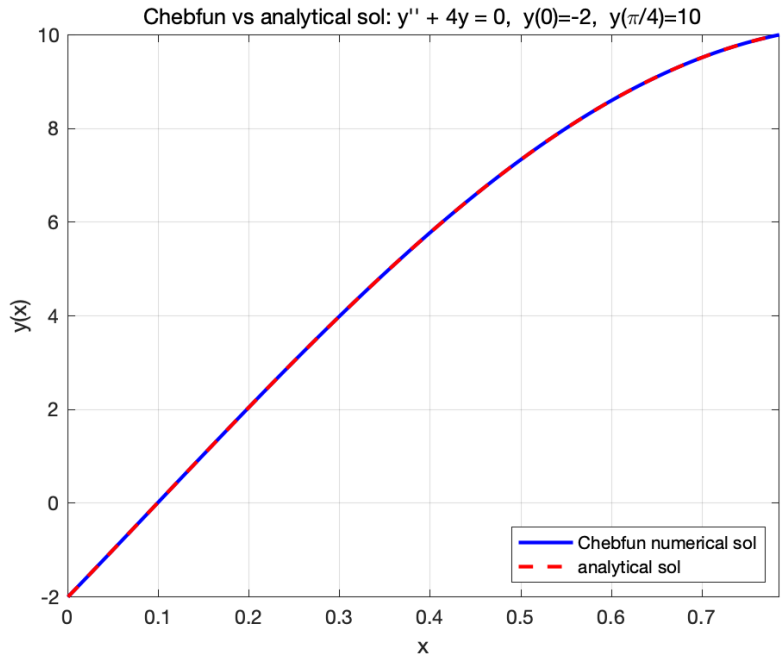

Figure 5: Chebfun solution against the exact analytical solution; the two curves are indistinguishable.

3.6 Example 6: Singular Sturm–Liouville problem

We solve the singular nonhomogeneous problem $-(xy')' = \mu xy + f(x)$ on $[0, 1]$ with $y(1) = 0$ and boundedness at $x = 0$, choosing $\mu = 10$ (not an eigenvalue) and $f(x) = \sin(\pi x)$ (fig. 6). The coefficient x vanishes at the left endpoint, so the operator is singular there.

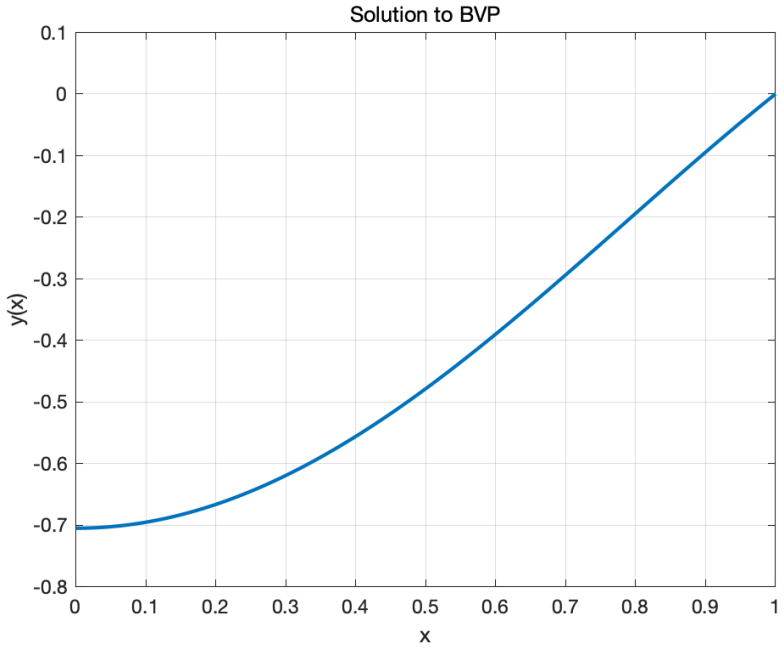

Figure 6: Solution of the singular Sturm–Liouville BVP.

3.7 Example 7: Lorenz attractor

Chebfun also solves IVPs. The Lorenz system, a classical example of deterministic chaos, is solved for three nearly identical initial conditions (differing by 10^{-6}). The trajectories track each other initially and then diverge, illustrating sensitive dependence on initial data (fig. 7).

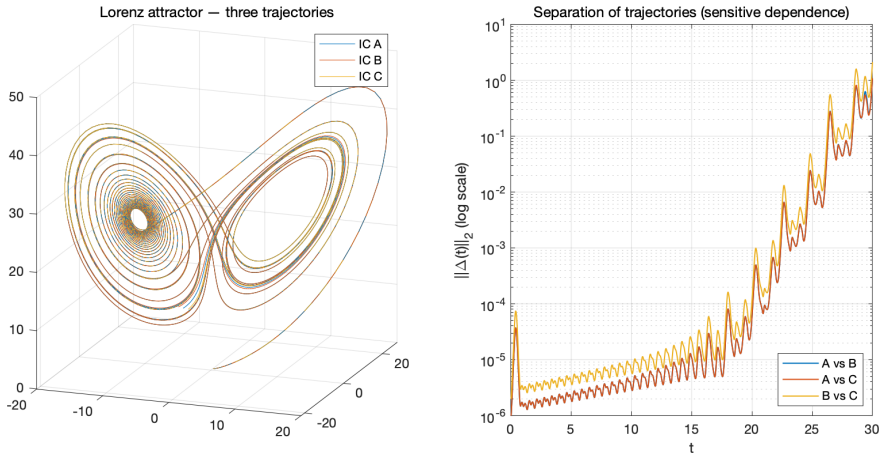

Figure 7: Lorenz attractor computed by Chebfun for three close initial conditions; the separation between trajectories grows exponentially.

3.8 Example 8: Predator–prey dynamics

The Lotka–Volterra model

$$x' = \alpha x - \beta xy, \qquad y' = -\gamma y + \delta xy,$$

with $(\alpha, \beta, \gamma, \delta) = (1.0,\ 0.1,\ 1.5,\ 0.075)$ and $x(0) = 10$, $y(0) = 5$, produces the familiar oscillations and closed phase-plane orbit (fig. 8).

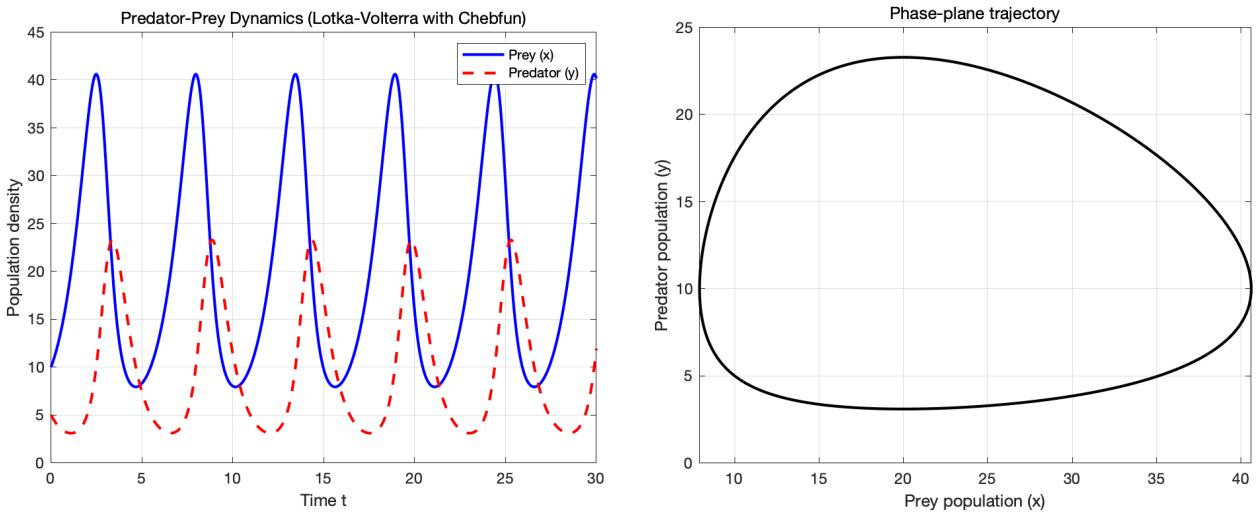

Figure 8: Predator–prey dynamics: time series (left) and closed phase-plane orbit (right).

4 Numerical Experiments

The examples above show that Chebfun *solves* these problems; this section measures *how well*. Each experiment isolates one property — convergence rate, eigenvalue accuracy, non-uniqueness, or adaptivity — and reports numbers produced by the accompanying scripts.

4.1 Spectral versus second-order convergence

We solve the problem of Example 5, whose exact solution is known, two ways: a standard second-order centered finite-difference scheme on grids of increasing size, and Chebfun's spectral collocation. Table 1 reports the maximum pointwise error and the empirical order of accuracy $\log_2(e_{N/2}/e_N)$.

Table 1: Finite-difference convergence for $y''+4y=0$. The empirical order approaches the theoretical $O(N^{-2})$. Chebfun reaches 7.9×10^{-14} with only 13 degrees of freedom.

N (finite diff.)	max error	order
8	5.790×10^{-3}	—
16	1.616×10^{-3}	1.84
32	4.290×10^{-4}	1.91
64	1.106×10^{-4}	1.96
128	2.807×10^{-5}	1.98
256	7.073×10^{-6}	1.99
512	1.775×10^{-6}	1.99
1024	4.446×10^{-7}	2.00
Chebfun (13 DOF)	7.905×10^{-14}	spectral

The finite-difference error decreases by a factor of about four per grid doubling, confirming second-order accuracy, but even at $N=1024$ it is only $\sim10^{-7}$. Chebfun attains machine precision with 13 coefficients (fig. 9).

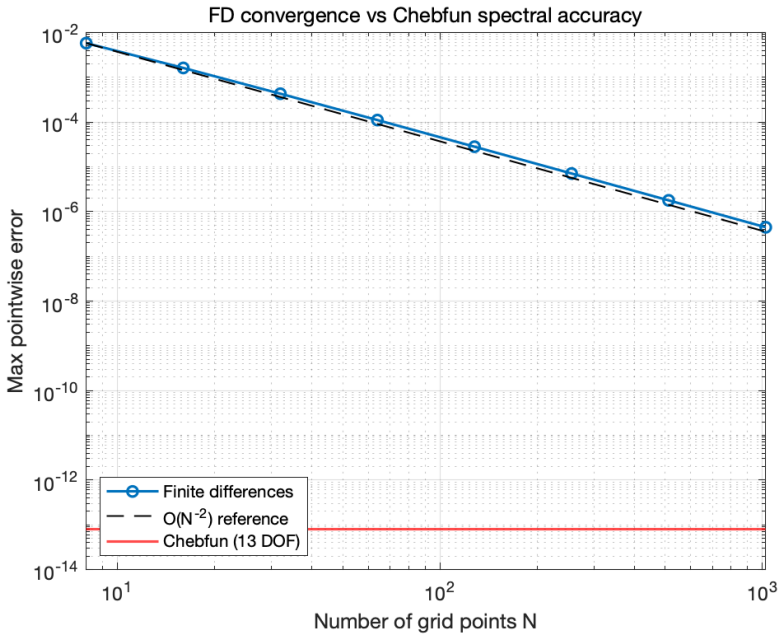

Figure 9: Finite-difference error versus the $O(N^{-2})$ reference line; the horizontal line marks Chebfun's machine-precision error.

4.2 Sturm–Liouville eigenvalues

We compute the first six eigenpairs of $-y'' = \lambda y$ with $y(0) = y(\pi) = 0$, whose exact eigenvalues are $\lambda_n = n^2$ with eigenfunctions $\sin(nx)$, using `eigs` on a `chebop`. All six eigenvalues are recovered to a few parts in 10^{12} (table 2), and the eigenfunctions match $\sin(nx)$ (fig. 10).

Table 2: Computed Sturm–Liouville eigenvalues versus the exact values $\lambda_n = n^2$.

n	computed	exact n^2	abs. error
1	1.0000000000	1	3.31×10^{-13}
2	4.0000000000	4	7.27×10^{-13}
3	9.0000000000	9	1.65×10^{-12}
4	16.000000000	16	1.98×10^{-12}
5	25.000000000	25	2.44×10^{-12}
6	36.000000000	36	1.18×10^{-12}

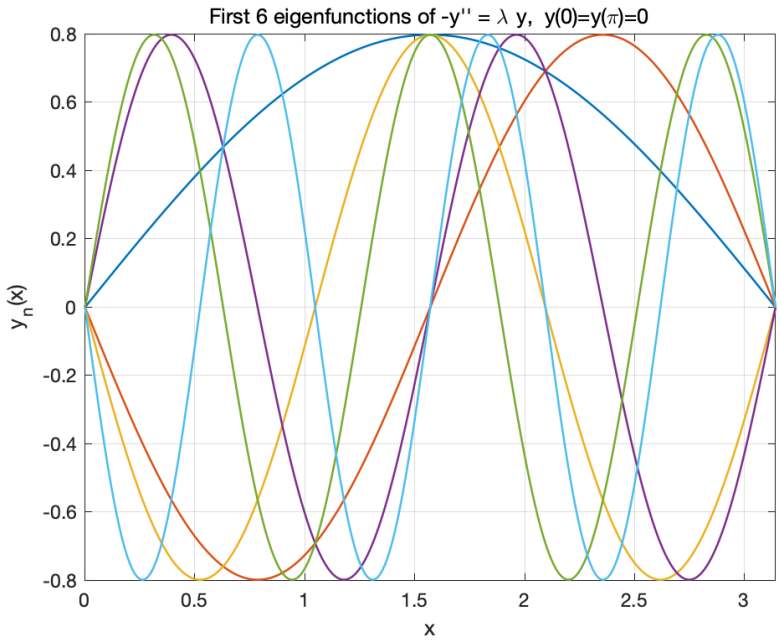

Figure 10: First six eigenfunctions of $-y'' = \lambda y$, $y(0) = y(\pi) = 0$.

4.3 Non-uniqueness: the Bratu problem

The Bratu problem $u'' + \lambda e^u = 0$, $u(0) = u(1) = 0$, has *two* solutions for $0 < \lambda < \lambda_c \approx 3.51$, which merge at a fold and then disappear. This is case (ii) of the existence discussion made concrete: Newton's method converges to a different branch depending on the initial guess. With $\lambda = 1$, starting from $u \equiv 0$ gives the lower branch ($\max u = 0.1405$), while starting from the tall bump

$u = 20x(1-x)$ gives the upper branch ($\max u = 4.0915$); see fig. 11.

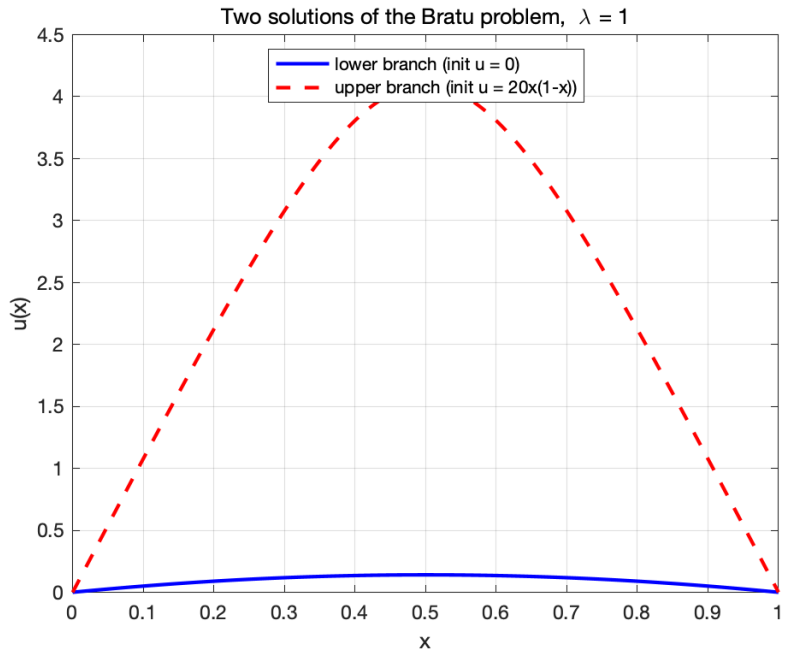

Figure 11: Two solutions of the Bratu problem at $\lambda = 1$, found from different Newton initial guesses.

4.4 Adaptivity: boundary layers

The singularly perturbed problem $\varepsilon u'' + u' = 1$, $u(0) = u(1) = 0$, has a boundary layer of width $O(\varepsilon)$ at $x = 0$, with exact solution $u(x) = x - (1 - e^{-x/\varepsilon})/(1 - e^{-1/\varepsilon})$. As ε shrinks, Chebfun automatically raises the polynomial degree to resolve the layer while keeping the error at machine precision (table 3, fig. 12).

Table 3: Chebfun's adaptive degree of freedom count and error as the boundary layer narrows. The cost grows roughly like $1/\sqrt{\varepsilon}$ while the accuracy stays near machine precision.

ε	Chebfun DOF	max error
10^{-1}	23	1.98×10^{-14}
10^{-2}	59	1.41×10^{-14}
10^{-3}	170	5.58×10^{-14}
10^{-4}	488	5.51×10^{-12}

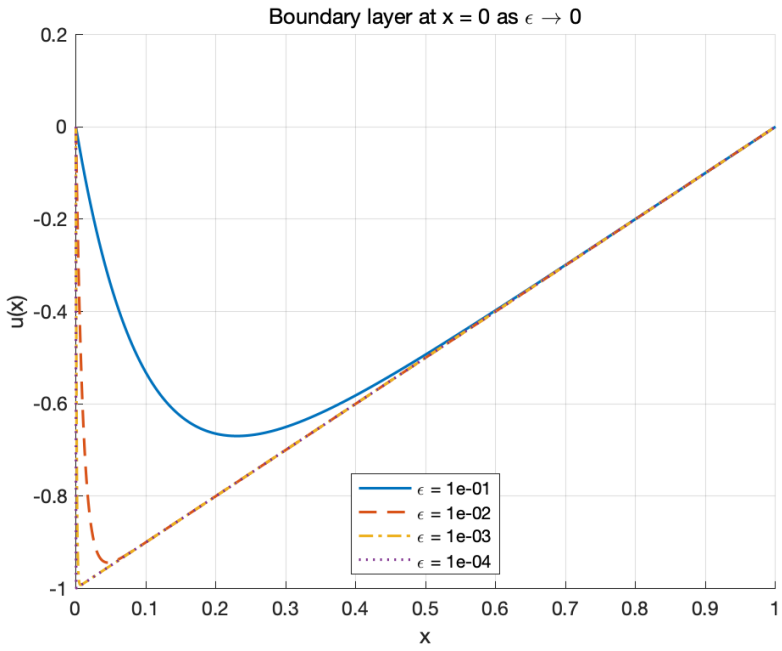

Figure 12: Boundary layer at $x = 0$ sharpening as $\varepsilon \to 0$.

5 Discussion and Conclusion

Across eight examples and four experiments, Chebfun combines the rigor of spectral methods with a remarkably simple interface. The `chebop` abstraction lets nonlinear operators, nonlinear boundary conditions, coupled systems, and singular problems be written almost exactly as they appear on paper, while the backslash hides the damped-Newton machinery underneath.

The experiments quantify the payoff. On a problem with a known exact solution, Chebfun reached 8×10^{-14} with 13 degrees of freedom, where a second-order finite-difference scheme needed more than a thousand grid points to reach 10^{-7} (table 1). Sturm–Liouville eigenvalues were recovered to parts in 10^{12} (table 2), the Bratu problem exposed genuine solution non-uniqueness controlled by the initial guess (fig. 11), and boundary layers down to width 10^{-4} were resolved automatically with a degree-of-freedom count that grew only as the layer demanded (table 3). Taken together, the adaptivity and high accuracy make Chebfun especially well suited to prototyping and research in applied mathematics, where correctness and ease of expression matter more than hand-tuned performance.

Acknowledgments

I would like to thank Prof. Antti Rasila for proposing the problems studied in this report, and Feitong Tu for collaborating on the collection of examples.

References

[1] W. E. Boyce and R. C. DiPrima. *Elementary Differential Equations and Boundary Value Problems*. Wiley.

[2] W. Han and K. E. Atkinson. *Theoretical Numerical Analysis: A Functional Analysis Framework*. Springer, 2009.

[3] L. N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2013.

[4] U. M. Ascher and L. R. Petzold. *Computer Methods for Ordinary Differential Equations and Differential-Algebraic Equations*. SIAM, 1998.

[5] T. A. Driscoll, N. Hale, and L. N. Trefethen. *Chebfun Guide*. Pafnuty Publications, 2014. https://www.chebfun.org.